Assignment 5:

Quicksort Algorithm: Implementation, Analysis, and Randomization

Hoai Nam Tran

(MSCS-532-M80) Algorithms and Data Structures

Dr. Michael Solomon

University of the Cumberlands

1. Introduction

Sorting is one of the most fundamental operations in computer science and plays a central role in data processing, searching, indexing, and optimization problems. Efficient sorting algorithms are critical in database systems, operating systems, search engines, and large-scale distributed systems. Among comparison-based sorting algorithms, Quicksort is widely regarded as one of the most efficient and practical algorithms in real-world applications.

Quicksort was introduced by Tony Hoare in 1960 and remains one of the most widely used internal sorting algorithms due to its excellent average-case performance and in-place memory usage (Hoare, 1961). Although its worst-case time complexity is $O(n^2)$, its expected time complexity is $O(n \log n)$, making it competitive with other divide-and-conquer algorithms, such as Merge Sort (Cormen et al., 2022).

2. GitHub Link Section

https://github.com/NamTran9694/MSCS532_Assignment5/blob/main/Deterministic_Randomized_Quicksort.py

3. Deterministic Quicksort Implementation and Analysis

3.1. Algorithm Overview

Quicksort is a divide-and-conquer algorithm that operates by:

- Selecting a pivot element.
- Partitioning the array into two subarrays based on the pivot.
- Recursively applying the same procedure to the subarrays.

The partitioning step ensures that elements smaller than the pivot are placed on its left, and larger elements on its right (Cormen et al., 2022).

In this implementation, the last element of the array is selected as the pivot, which is a common textbook strategy.

3.2.Theoretical Performance Analysis

- **Best Case Time Complexity:** The best case occurs when the pivot consistently divides the array into two equal halves. The recurrence relation is: $T(n)=2T(n/2)+O(n)$. Applying the Master Theorem yields: $T(n)=O(n\log n)$. Balanced partitioning minimizes recursion depth and ensures logarithmic growth (Goodrich et al., 2014).
- **Average Case Time Complexity:** The average-case complexity considers all possible pivot positions. When pivot selection is random or evenly distributed, partitions are reasonably balanced on average. The expected recurrence becomes: $T(n)=T(k)+T(n-k-1)+O(n)$. Averaging over all possible values of k leads to an expected time complexity of: $O(n \log n)$. This expected bound makes Quicksort highly efficient in practical applications (Sedgewick & Wayne, 2011).
- **Worst Case Time Complexity:** The worst case occurs when the pivot is always the smallest or largest element, resulting in highly unbalanced partitions. This happens when: The input array is already sorted. The input is reverse sorted. A poor deterministic pivot strategy is used. The recurrence becomes: $T(n)=T(n-1)+O(n)$. Which expands to: $O(n^2)$. This quadratic growth significantly reduces performance on large inputs (Cormen et al., 2022).

3.3.Space Complexity Analysis

Quicksort is considered an in-place sorting algorithm because it does not require auxiliary arrays for merging. However, recursive calls require stack space:

- Best/Average case recursion depth: O(log n).
- Worst case recursion depth: O(n)

Thus, while Quicksort is memory-efficient compared to Merge Sort, poor pivot selection increases stack overhead (Goodrich et al., 2014).

4. Randomized Quicksort

4.1. Motivation for Randomization

Deterministic pivot selection makes Quicksort vulnerable to adversarial inputs. If the input is already sorted, selecting the last element as pivot consistently produces unbalanced partitions. Randomized Quicksort addresses this issue by selecting the pivot uniformly at random from the subarray (Cormen et al., 2022).

4.2. Theoretical Impact of Randomization

Randomization does not change the worst-case time complexity of Quicksort, which remains O (n^2). However, it significantly reduces the probability of encountering this worst case. With random pivot selection:

- The expected running time becomes O(nlogn) for all inputs.
- Performance becomes independent of input order.
- The algorithm becomes robust against adversarial sequences.

Randomization is a common technique in algorithm design to ensure expected efficiency even when worst-case inputs exist.

5. Empirical Analysis

```
==================================================
Input Size: 1000
==================================================
Random Data → Deterministic: 0.00095s | Randomized: 0.00122s
Sorted Data → Deterministic: 0.03734s | Randomized: 0.00110s
Reverse Data → Deterministic: 0.02631s | Randomized: 0.00140s

==================================================
Input Size: 5000
==================================================
Random Data → Deterministic: 0.00578s | Randomized: 0.00776s
Sorted Data → Deterministic: 1.02340s | Randomized: 0.00913s
Reverse Data → Deterministic: 0.60206s | Randomized: 0.00729s

==================================================
Input Size: 10000
==================================================
Random Data → Deterministic: 0.01264s | Randomized: 0.01616s
Sorted Data → Deterministic: 3.93921s | Randomized: 0.01358s
Reverse Data → Deterministic: 1.87880s | Randomized: 0.01453s
```

5.1.Analysis Results

The experimental results highlight the impact of pivot selection on Quicksort performance.

For **random data**, both deterministic and randomized Quicksort perform similarly, showing efficient behavior consistent with the expected O(n log n) time complexity. The deterministic version is slightly faster due to lower overhead.

For **sorted data**, deterministic Quicksort shows a dramatic increase in execution time as input $O(n^2)$, caused by highly unbalanced partitions when the last element is always chosen as the pivot. In contrast, randomized Quicksort maintains stable performance across all input sizes.

For **reverse-sorted data**, deterministic Quicksort also experiences significant slowdown, though slightly less severe than the sorted case. Randomized Quicksort again remains consistent and efficient.

Overall, deterministic Quicksort is sensitive to input order, while randomized Quicksort provides more stable performance. These results align with theoretical expectations and demonstrate the effectiveness of random pivot selection in avoiding worst-case scenarios.

5.2.Key Observations

- Deterministic Quicksort performs well only when input data is randomly distributed.
- Sorted and reverse-sorted inputs trigger worst-case behavior in deterministic Quicksort.
- Randomized Quicksort significantly reduces sensitivity to input distribution.
- The overhead of random pivot selection is minimal compared to the performance gains.
- Empirical results strongly validate theoretical complexity analysis.

5.3.Conclusion of Empirical Study

The empirical analysis confirms that while deterministic Quicksort can be efficient in favorable conditions, it is highly vulnerable to specific input patterns. Randomized Quicksort provides a more reliable alternative by ensuring consistent performance across different datasets.

These findings highlight the importance of pivot selection strategy in practical algorithm design and demonstrate how randomization can improve robustness without increasing asymptotic complexity.

6. Practical Application

Quicksort is widely used in:

- Standard library implementations (e.g., hybrid variants)
- Database indexing

- In-memory sorting systems
- High-performance computing

Modern implementations often combine Quicksort with other techniques (such as insertion sort for small subarrays or introsort) to mitigate worst-case behavior (Sedgewick & Wayne, 2011).

7. Conclusion

This assignment explored deterministic and randomized Quicksort implementations through theoretical and empirical analysis. The results confirm:

- Deterministic Quicksort achieves excellent average-case performance but is vulnerable to worst-case inputs.
- Randomized pivot selection dramatically reduces the likelihood of worst-case behavior.
- Empirical observations support theoretical complexity analysis.
- Randomization enhances robustness without altering asymptotic bounds.

Understanding these trade-offs is critical when designing efficient and scalable systems, particularly in environments where input distributions are unpredictable.

8. Reference

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). Data structures and algorithms in Python. Wiley.

Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.

9. Python Code

```python
"""
Assignment 5 – Quicksort Implementation, Analysis, and Randomization
Author: Hoai Nam Tran
Course: MSCS-532 – Algorithms and Data Structures

----------------------------------------------------------------------
Theoretical Complexity:

Deterministic Quicksort:
    Best Case:    O(n log n)
    Average Case: O(n log n)
    Worst Case:   O(n^2)

Randomized Quicksort:
    Expected Time: O(n log n)
    Worst Case:    O(n^2) (extremely unlikely)

Space Complexity:
    Average recursion depth: O(log n)
    Worst recursion depth:   O(n)
----------------------------------------------------------------------
"""

import random
import time


# ==========================================================
# 1. Deterministic Quicksort
# ==========================================================

def partition(arr, low, high):
    # Choose pivot: In deterministic quicksort, choose the last element as the
pivot.
    pivot = arr[high]

    # Mark the boundary between: elements <= pivot   |   elements > pivot
    i = low - 1

    # Rearrange elements
    for j in range(low, high):

        # If current element is less than or equal to pivot
```

```
        if arr[j] <= pivot:
            i += 1  # Move boundary forward
            arr[i], arr[j] = arr[j], arr[i]   # Swap smaller element into correct
position

    # Place pivot in correct sorted position
    arr[i + 1], arr[high] = arr[high], arr[i + 1]

    # Return pivot index
    return i + 1


def quicksort(arr, low, high):
    """
    RECURSIVE QUICKSORT FUNCTION
    This function applies divide-and-conquer:
        1. Partition array
        2. Recursively sort left side
        3. Recursively sort right side
    Base Case:
        If subarray has 0 or 1 element, it is already sorted.
    Parameters:
        arr  : list to be sorted
        low  : starting index
        high : ending index
    """

    # Base case condition
    while low < high:
        # Partition array and get pivot index
        pi = partition(arr, low, high)

        # Recurse into smaller partition to keep stack shallow
        if (pi - low) < (high - pi):
            quicksort(arr, low, pi - 1) # Recursively sort elements smaller than
pivot
            low = pi + 1
        else:
            quicksort(arr, pi + 1, high) #Recursively sort elements greater than
pivot
            high = pi - 1


# ==========================================================
# 2. Randomized Quicksort
```

```python
# ============================================================

def randomized_partition(arr, low, high):
    """
    RANDOMIZED PARTITION
    Instead of always choosing the last element as pivot, we randomly select an
element between low and high.
    Strategy:
        1. Pick random index
        2. Swap it with last element
        3. Call standard partition()
    """

    # Select random index within subarray
    random_index = random.randint(low, high)

    # Swap random element with last element (pivot position)
    arr[random_index], arr[high] = arr[high], arr[random_index]

    # Perform standard partition
    return partition(arr, low, high)


def randomized_quicksort(arr, low, high):
    """
    RANDOMIZED QUICKSORT
    Same recursive structure as deterministic version, but uses randomized pivot
selection.
    """

    if low < high:

        # Partition using randomized pivot
        pi = randomized_partition(arr, low, high)

        # Recursively sort left partition
        randomized_quicksort(arr, low, pi - 1)

        # Recursively sort right partition
        randomized_quicksort(arr, pi + 1, high)


# ============================================================
# 3. Performance Testing
# ============================================================
```

```python
def test_algorithm(sort_function, arr):

    start_time = time.time()

    # Execute sorting algorithm
    sort_function(arr, 0, len(arr) - 1)

    end_time = time.time()

    return end_time - start_time


def run_performance_tests():

    sizes = [1000, 5000, 10000]

    for size in sizes:

        print("\n====================================================")
        print(f"Input Size: {size}")
        print("====================================================")

        #Generate different input distributions
        random_data = [random.randint(0, 10000) for _ in range(size)]
        sorted_data = sorted(random_data)
        reverse_data = sorted(random_data, reverse=True)

        for dataset_name, dataset in [
            ("Random", random_data.copy()),
            ("Sorted", sorted_data.copy()),
            ("Reverse", reverse_data.copy())
        ]:

            # Measure deterministic version
            det_time = test_algorithm(quicksort, dataset.copy())

            # Measure randomized version
            rand_time = test_algorithm(randomized_quicksort, dataset.copy())

            print(
                f"{dataset_name} Data → "
                f"Deterministic: {det_time:.5f}s | "
                f"Randomized: {rand_time:.5f}s"
            )
```

```python
# ==========================================================
# Main Execution Block
# ==========================================================

if __name__ == "__main__":

    # Demonstration with small example
    example_array = [10, 7, 8, 9, 1, 5]

    print("Original array:", example_array)

    quicksort(example_array, 0, len(example_array) - 1)

    print("Sorted array (Deterministic):", example_array)

    # Run empirical comparison
    run_performance_tests()
```